

FMU Análise e Desenvolvimento de sistemas

APS – ATIVIDADES PRÁTICAS SUPERVISIONADAS

Nome: Lucas Souza Novais

3º semestre

RA: 2025113256

Disciplina: Programação Orientada a Objetos

Professor: Luciano Tadeu

São Paulo – SP 2025

Código-Fonte - Sistema de Gerenciamento de Biblioteca Digital

Usuario.java

```
public abstract class Usuario {  
  
    protected String nome;  
  
    protected int id;  
  
    protected String senha;  
  
  
    public Usuario(String nome, int id, String senha) {  
  
        this.nome = nome;  
  
        this.id = id;  
  
        this.senha = senha;  
  
    }  
  
  
    public abstract void emprestar(Acervo item);  
  
    public void devolver(Acervo item) {  
  
        System.out.println("Item devolvido: " + item.getTitulo());  
  
    }  
}
```

Leitor.java

```
import java.util.ArrayList;  
  
  
public class Leitor extends Usuario {  
  
    private ArrayList<Acervo> emprestimos = new ArrayList<>();  
  
  
    public Leitor(String nome, int id, String senha) {  
  
        super(nome, id, senha);  
  
    }  
}
```

```
}
```

```
@Override
```

```
public void emprestar(Acervo item) {
```

```
    if (emprestimos.size() < 3) {
```

```
        emprestimos.add(item);
```

```
        System.out.println("Item emprestado: " + item.getTitulo());
```

```
    } else {
```

```
        System.out.println("Limite de empréstimos atingido.");
```

```
    }
```

```
}
```

```
}
```

Bibliotecario.java

```
public class Bibliotecario extends Usuario {
```

```
    public Bibliotecario(String nome, int id, String senha) {
```

```
        super(nome, id, senha);
```

```
    }
```

```
@Override
```

```
public void emprestar(Acervo item) {
```

```
    System.out.println("Bibliotecário não realiza empréstimos.");
```

```
}
```

```
public void cadastrarItem(Acervo item) {
```

```
    System.out.println("Item cadastrado: " + item.getTitulo());
```

```
}
```

```
}
```

Acervo.java

```
public abstract class Acervo {  
  
    protected int codigo;  
  
    protected String titulo;  
  
  
    public Acervo(int codigo, String titulo) {  
  
        this.codigo = codigo;  
  
        this.titulo = titulo;  
  
    }  
  
  
    public String getTitulo() {  
  
        return titulo;  
  
    }  
  
  
    public abstract String info();  
  
}
```

Livro.java

```
public class Livro extends Acervo {  
  
    private String autor;  
  
    private int paginas;  
  
  
    public Livro(int codigo, String titulo, String autor, int paginas) {  
  
        super(codigo, titulo);  
  
        this.autor = autor;  
  
        this.paginas = paginas;  
  
    }  
  
}
```

```
}
```

```
@Override
```

```
public String info() {
```

```
    return "Livro: " + titulo + ", Autor: " + autor + ", Páginas: " + paginas;
```

```
}
```

```
}
```

Revista.java

```
public class Revista extends Acervo {
```

```
    private int edicao;
```

```
    private String mes;
```

```
    public Revista(int codigo, String titulo, int edicao, String mes) {
```

```
        super(codigo, titulo);
```

```
        this.edicao = edicao;
```

```
        this.mes = mes;
```

```
}
```

```
@Override
```

```
public String info() {
```

```
    return "Revista: " + titulo + ", Edição: " + edicao + ", Mês: " + mes;
```

```
}
```

```
}
```

DVD.java

```
public class DVD extends Acervo {
```

```
    private int duracao;
```

```
public DVD(int codigo, String titulo, int duracao) {  
    super(codigo, titulo);  
    this.duracao = duracao;  
}  
  
@Override  
public String info() {  
    return "DVD: " + titulo + ", Duração: " + duracao + " min";  
}  
}
```

Main.java

```
public class Main {  
    public static void main(String[] args) {  
        Leitor leitor = new Leitor("João", 1, "senha123");  
        Livro livro = new Livro(101, "Java Básico", "Autor X", 300);  
        Revista revista = new Revista(102, "TechHoje", 5, "Maio");  
        DVD dvd = new DVD(103, "Documentário", 90);  
  
        leitor.emprestar(livro);  
        leitor.emprestar(revista);  
        leitor.emprestar(dvd);  
  
        leitor.devolver(livro);  
    }  
}
```

Sistema de Gerenciamento de Biblioteca Digital

1. Introdução

Este projeto tem como objetivo implementar um sistema de gerenciamento de biblioteca digital utilizando os principais conceitos de programação orientada a objetos. O sistema simula a interação entre diferentes tipos de usuários e itens do acervo, incluindo funcionalidades como cadastro, empréstimos e devoluções.

2. Estrutura de Classes e Relações

2.1 Classe Usuario (abstract)

Classe base abstrata que representa um usuário da biblioteca. Contém atributos como nome, ID e senha com encapsulamento. Possui métodos de autenticação, empréstimo (abstrato), devolução e consulta de itens emprestados.

2.2 Classe Leitor

Subclasse de Usuario que representa um usuário comum da biblioteca. Implementa o método emprestar com uma restrição de até 3 itens por usuário.

2.3 Classe Bibliotecario

Subclasse de Usuario que representa um bibliotecário, com permissão para cadastrar novos itens ao acervo. Não realiza empréstimos.

2.4 Classe Acervo (abstract)

Classe abstrata que representa um item do acervo da biblioteca. Contém os atributos comuns a todos os tipos de acervo, como título e código. Define um método abstrato info() que será sobrescrito pelas subclasses.

2.5 Classe Livro

Subclasse de Acervo que representa um livro. Adiciona os atributos autor e número de páginas. Sobrescreve o método info().

2.6 Classe Revista

Subclasse de Acervo que representa uma revista. Adiciona os atributos edição e mês. Sobrescreve o método info().

2.7 Classe DVD

Subclasse de Acervo que representa um DVD. Adiciona o atributo duração. Sobrescreve o método info().

2.8 Classe Main

Classe principal que executa a lógica de teste do sistema, instanciando objetos, cadastrando itens, realizando empréstimos e imprimindo os resultados no console.

3. Introdução Teórica à Programação Orientada a Objetos

A Programação Orientada a Objetos (POO) é um paradigma que organiza o software em torno de objetos, ao invés de ações. Os objetos são instâncias de classes, que encapsulam dados (atributos) e comportamentos (métodos). Os principais pilares da POO são:

- **Encapsulamento**: restringe o acesso direto aos dados internos de um objeto;
- **Herança**: permite que uma classe herde características de outra classe;
- **Polimorfismo**: permite que objetos se comportem de diferentes formas com base no seu tipo real;
- **Abstração**: foca nos comportamentos essenciais, ocultando detalhes de implementação.

O uso desses princípios permite a construção de sistemas mais organizados, reutilizáveis e de fácil manutenção.

4. Código-Fonte Comentado

4.1 Usuario.java

Classe abstrata que define os atributos comuns de todos os usuários, como nome, ID e senha. Inclui os métodos get/set e um método abstrato para empréstimo, que será implementado pelas subclasses.

4.2 Leitor.java

Implementa a lógica de empréstimo com um limite de 3 itens. Herda da classe Usuario.

4.3 Bibliotecario.java

Representa um bibliotecário que pode cadastrar novos itens no sistema. Não realiza empréstimos.

4.4 Acervo.java

Classe abstrata que representa um item do acervo. Possui atributos básicos e define o método info().

4.5 Livro.java, Revista.java e DVD.java

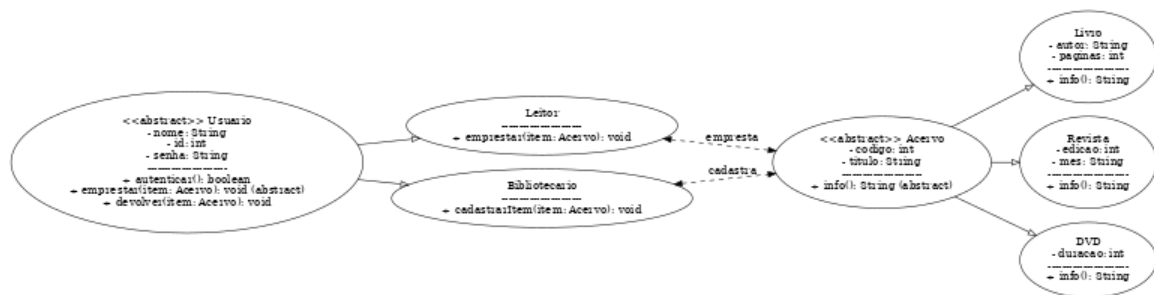
Subclasses de Acervo que implementam os atributos e comportamento específicos de cada tipo de item. Sobrescrevem o método info().

4.6 Main.java

Classe principal que instancia os objetos, simula empréstimos e exibe os resultados. Funciona como teste para validar o funcionamento do sistema.

5. Diagrama UML das Classes

A seguir, apresentamos o diagrama UML que representa a estrutura de classes do sistema. O diagrama evidencia as relações de herança entre as entidades, bem como associações entre os usuários e os itens do acervo.



6. Conclusão

O projeto proposto permite a aplicação prática dos principais conceitos da programação orientada a objetos, como herança, polimorfismo, encapsulamento e uso de coleções. A modelagem baseada em uma biblioteca digital oferece um contexto realista e aplicável ao cotidiano.